

AI 软件开发平台

从零启动、架构设计 与扩展实施手册

FastapiAdmin • Vue 3 • uv • PostgreSQL
LangGraph • Temporal • OpenSpec • C4

交付边界

在线组装源代码基础工程。已包含实际实现与本地契约测试；不是完全离线发行版，也未在本次环境通过真实上游整栈联调。

阅读导航

先跑通固定模板与干净交付，再扩展模型、框架和隔离执行器。

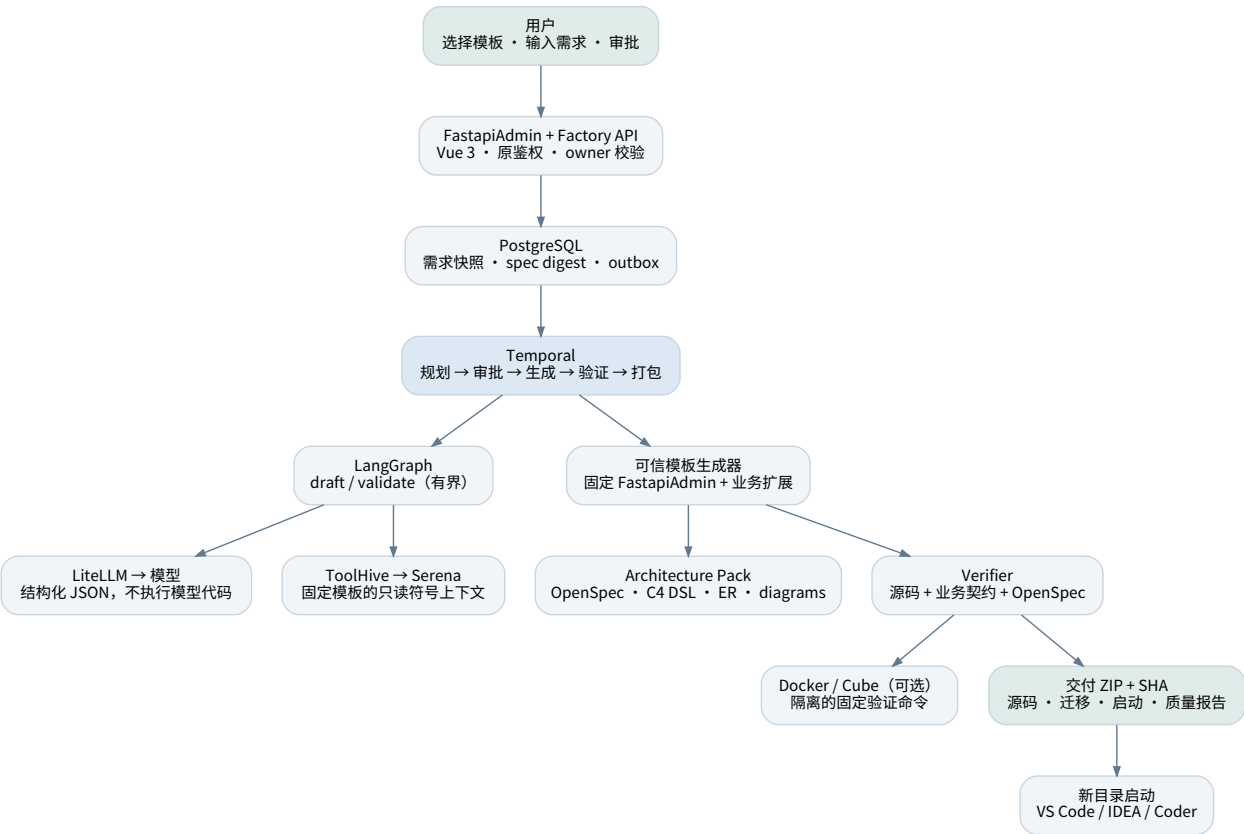
从零启动与首次验收	4
架构、职责与数据流	9
图表与架构包的使用	14
如何增加框架、业务能力和 Agent	16
运行、安全与运维	20
故障排查：先定位哪一段失败	23
可以直接交给编程 AI 的任务提示词	25
验证报告 · 本次交付范围	30
外部工具接入附录	33
官方来源与版本核对	37

源码 ZIP 内同时包含分章节 Markdown、完整 Markdown、测试报告及集成脚本。PDF 命令须根据你的实际路径替换占位符。

基础工程 0.1.0 • Windows / AMD x64 • uv / PostgreSQL / Vue

编制基线：2026-09-08。

交付类型：在线组装源代码基础工程。包含真实实现与本地契约测试，但不是完全离线发行版，也未在本次环境通过真实上游整栈联调。先阅读验证范围，再按步骤在本机验收。



阅读顺序

第一次使用先读“从零启动”；开始二次开发读“架构”和“如何增加框架”；接外部工具读对应集成附录；让编程AI协助时使用分任务提示词；准备交付前逐条检查验证报告与安全章节。

从零启动与首次验收

1. 你拿到的到底是什么

把这个目录理解为“建造平台的原料和装配说明”。Dockerfile 会下载你指定的真正 FastapiAdmin 固定提交，把新增研发平台代码接到它的应用工厂和 Vue 路由。它没有把一个极简 FastAPI 页面冒充完整 FastapiAdmin，也没有把上游 Agno 聊天功能当成整套软件研发流水线。

但是，本包 **不是完全离线包**，本次环境 **没有实际启动 Docker 整栈**。因此请把首次本机验收视为必要步骤。若你要求“下载即离线运行、11 个工具全部已联调、任意需求都生成完整产品”，本包尚未达到这一标准。

2. Windows + AMD 的准备工作

推荐 Windows + WSL2 Ubuntu + Docker Desktop，源码放在 Ubuntu 的 `~/src`，而不是长期从 `/mnt/c` 的 Windows 文件夹运行大量 Linux 构建。AMD 指处理器架构，不代表一定有 GPU，也不代表自动具备 KVM。模型可以放远端；不要求本机显卡。[S13][S14]

第一步，在 Windows 任务管理器的“性能 → CPU”查看虚拟化是否已启用。未启用时按主板说明进入 BIOS 开启 SVM/AMD-V。管理员 PowerShell 执行：

```
wsl --install -d Ubuntu-24.04
wsl --update
wsl --list --verbose
```

如果 Ubuntu 已经安装，不重复安装，确认 VERSION 一列为 2。首次打开 Ubuntu 会要求设置 Linux 用户名和密码；输入密码时没有星号是正常现象。

第二步，安装 Docker Desktop，选择 WSL2 后端，在 Settings → Resources → WSL Integration 中启用你的 Ubuntu。打开 Ubuntu 终端执行：

```
docker version
docker compose version
```

成功时应同时看到 Docker Client 和 Server。只有 Client 而 Server 连接失败，说明 Docker Desktop 未启动或 WSL 集成未启用。不要在同一 WSL 里盲目再安装另一套 dockerd。

第三步，准备终端工具并解压。把下面 Windows 路径换成你真实下载位置：

```
sudo apt-get update
sudo apt-get install -y git unzip python3 ca-certificates
mkdir -p ~/src
cd ~/src
unzip /mnt/c/Users/你的Windows用户名/Downloads/AI_RND_Platform_Foundation_0.1.0.zip
cd ai-rnd-foundation
ls
```

应能看见 `Dockerfile`、`compose.yaml`、`factory`、`scripts`、`docs`。终端中的当前目录很重要：后文未特别注明的命令，都在这个项目根目录运行。

你可以在 Windows 安装 VS Code 及 WSL 扩展，在 Ubuntu 根目录执行 `code .` 打开整个项目。先不用理解全部源码。

3. 第一次只运行核心流程

暂不安装 Cube、Coder、ToolHive，也不填模型密钥。先验证基础工程，避免 11 个服务同时报错。

```
python3 scripts/doctor.py
python3 scripts/init_env.py
docker compose config --quiet
docker compose up --build -d
```

`init_env.py` 只在 `.env` 不存在时生成随机 PostgreSQL、Redis 和 JWT 密钥，不覆盖已有配置。`.env` 是本地秘密，不能上传 Git、聊天截图或发给编程 AI。`docker compose config --quiet` 只检查配置；不要把展开的 `docker compose config` 输出公开，因为它包含密码。

也可以用 `bash scripts/start.sh` 执行初始化和启动。这个脚本不是隐藏式安装器，你可以先打开查看内容。

首次构建依次进行：拉取固定上游 → 组装新增 Vue 页面 → pnpm 安装和类型检查 → 前端构建 → uv 解析平台与上游共同依赖 → 安装 → 启动 PostgreSQL、Redis、Temporal → 平台迁移 → API/worker 启动。任一步失败，必须保留并检查失败位置；不能删除类型检查、跳过迁移或改成 demo 后就宣称已修好。

```
docker compose ps -a
docker compose logs --tail=120 migrate api worker temporal
```

`init-data`、`migrate` 正常完成后显示 `exited(0)` 是正常的，它们是一次性任务。API/worker/PG/Redis/Temporal 应持续运行。健康检查只覆盖对应服务，API health 成功不意味着 worker 已执行任务。

4. 登录并找到新增研发界面

浏览器打开 `http://localhost:8000/web/`。上游当前 `README` 的本地快速启动说明使用 `admin / 123456`；首次登录后立即改密码，删除不需要的演示用户。原始模板的账号初始化逻辑未在本次环境实际运行，若登录失败查看 API 初始化日志和下载的上游说明，不尝试猜测数据库密码。[S01]

登录后地址栏输入：

```
http://localhost:8000/web/#/factory
```

这是新增的独立研发页面，默认没有自动写入 FastapiAdmin 菜单数据库。以后可按上游菜单管理方式加一个可见菜单入口。`/web/#/factory` 中 `#` 后面是浏览器路由，不是后端文件路径。

页面使用原来的登录令牌，后端复用 JWT、Redis 会话以及实时用户检查。不要把 `FACTORY_TOKEN` 粘进去；它只用于可选诊断应用，不是主界面的登录凭据。

5. 先跑明确标识的 demo

填写项目名称“设备台账演示”，需求填写“建设设备与维修记录管理，验证台账和关联记录”。选择唯一已实现的 `fastapiadmin-pg-v1` 模板；provider 选择 demo，sandbox 选择 static，暂不勾 Serena。

demo 总是生成同一套设备和维修记录结构。它的作用是排除模型差异、验证装配链，不会把你输入的任意软件需求真正实现。页面会展示该限制，必须主动认可才能批准。

点击启动后，状态顺序应为：

```
QUEUED → PLANNING → AWAITING_APPROVAL  
→ GENERATING → VERIFYING → PACKAGING → READY
```

在审批页检查表、字段、引用和未实现项。确认后点击批准；不接受就拒绝，补充需求后新建一个 run。当前不支持直接在审批页编辑 JSON。后补的聊天消息不会修改正在运行任务的需求快照。

READY 表示 **通过本版门禁的源码骨架可下载**，不是生产验收合格。点击下载，保存 ZIP，打开其中的 `delivery/quality.json`。如果状态 FAILED，先看事件和 worker 日志，不重复点几十次创建任务。

6. 在干净目录启动下载的产品

把下载 ZIP 解压到一个新目录，不要覆盖平台自身目录。这是“被生成的软件”，与“生成软件的平台”是两个独立项目。

```
cd ~/src/你解压后的产品目录
python3 scripts/init_product.py
docker compose config --quiet
docker compose up --build -d
docker compose logs --tail=100 app
```

产品默认网页 `http://localhost:8010/web/`；登录并改密码后打开 `http://localhost:8010/web/#/business`。先新增一个设备，记住 ID，再新增引用它的维修记录。关联字段目前填写父记录 ID，并非智能下拉选项。

至少验收：刷新后数据仍在；重启 app 后数据仍在；第二个用户看不到第一个用户的业务记录；跨用户引用被拒绝；有子记录的父记录不能直接删除。不要因为看到了表单就认为数据库或权限已验收。

平台与产品默认端口不同。运行多个产品时，为每个产品修改 Compose 项目名和端口，不能共用同一组数据卷或照抄密码。

7. 让真实需求进入模型

只有先跑通上面的固定 demo，才接真实模型。已有 Ollama 时，在其所在机器运行 `ollama list`，复制一个确实已安装、支持聊天并能稳定输出 JSON 的模型名。不要照抄手册中的占位符，也不要凭显存大小假定任意模型都能运行。

编辑平台根目录 `.env` 的 `OLLAMA_MODEL`，然后执行：

```
python3 scripts/configure_model.py
docker compose --profile ai up -d --force-recreate api worker litellm
docker compose exec worker python scripts/probe_integrations.py model
```

脚本为 LiteLLM 生成 `factory-planner` 路由，将平台模型密钥设置为网关 master key。默认网关访问 `http://host.docker.internal:11434` 的 Ollama；本机部署位置和绑定地址不同时需要修改 `integrations/litellm/config.yaml`。不要为了 Docker 连通而把无认证 Ollama 暴露公网。[S09]

探针成功后在网页选择 `litellm`，输入一个小而具体的需求，例如：“仓库货架管理，货架有名称、区域、启用状态；物品有名称、数量、所属货架，只做台账 CRUD，不做出入库交易。”模型产出的字段应与本次需求一致，不应仍然固定出现维修记录。

数量约束、唯一编码、库存扣减事务等不属于本版自动完成能力。模型应将未支持项列出；你仍需人工检查它有没有遗漏。模型返回 400、超时或非法 JSON 时会失败，不偷偷切换到 demo。

8. 开关服务、记录版本

```
# 普通停止，保留数据库：
docker compose stop
# 继续：
docker compose up -d
# 编辑 .env 后：
docker compose up -d --force-recreate api worker
# 成功构建后记录真实依赖：
bash scripts/capture_locks.sh
```

不要把 `restart` 当成重新载入 Compose 环境变量；修改 `.env` 通常需要 `recreate`。不要把 `down -v` 当成日常停止。升级前备份 `.env`、PostgreSQL、产物目录和 Temporal 开发数据，并实际演练恢复。

架构、职责与数据流

1. 产品目标：先建立可信的交付闭环

我们要建设的是“软件工厂”，不是单纯聊天窗口，也不是把十一个开源项目同时启动起来。用户需求必须转换为明确规格；模板必须可追溯；生成动作必须受限；质量证据必须能够复查；交付包必须离开平台独立运行。

本版刻意收窄到 Python/FastAPI + Vue3 + PostgreSQL 的管理型应用。固定模板保留上游管理框架，扩展独立业务 schema、迁移、CRUD 路由和动态 Vue 业务页面。它并未自动生成每个模块完整的上游 controller/service/crud/model 四层文件；当前业务扩展采用一个可阅读、可测试的类型化运行时。下一阶段可换成四层源码 renderer，而不必推翻平台工作流。

原规划强调 Golden Template、Architecture Pack、Provider、确定性验收，这些保留。默认 GitHub PR、并行集群和必须先有 Coder/Cube 的顺序被取消。先下载可运行产品，再增加代码托管、自由编码和多人协作。[U01]

2. 五个职责区域

用户入口是 FastapiAdmin 管理壳及新增 Vue 页面。原来的用户、角色、登录和会话不重写。平台研发功能放在 /factory-api，产品业务功能放在 /business-api，避免与上游版本化 API 命名冲突。

平台控制层是 factory/api.py、Repository、PostgreSQL、事务 outbox。它记录项目、消息、运行、审批、事件和产物路径，不在 HTTP 请求里直接运行几分钟的 AI 工作。

编排层由 Temporal 和 LangGraph 分工。Temporal 处理跨阶段的可靠执行、等待、重试和状态恢复；LangGraph 在一次规划 activity 内完成 draft/validate 的有界循环，不再建立第二套持久任务调度系统。[S03][S04]

生成与验证层用固定提交的模板、确定性业务 renderer、OpenSpec 文档、Graphviz/diagrams、源码与业务契约测试。模型不能直接写任意文件、数据库 URL、Dockerfile 或 shell 命令。这是本版安全边界，不是为了装饰而加入“多 Agent”。

开发与隔离层为可选 Coder、Docker verifier、Cube。Coder 保存长期 IDE 工作；Cube 提供临时 MicroVM；Docker verifier 是本地测试的受限命令运行器。它们不是同一个文件系统，更不能默认共享主机全部目录。[S06][S07]

3. 逻辑关系图



architecture/workspace.dsl 是平台的 C4 源文件；生成产品中的同名文件描述该产品，两者不能混为一张图。 architecture/platform.svg 是本包绘制的逻辑关系图，不是 Structurizr 已完成解析的证明。

4. 工具职责与当前接入程度

工具	在项目中的工作	本包实现	本次真实运行情况
FastapiAdmin	平台管理壳、产品 Golden Template	固定提交组装、复用鉴权、追加 Vue 路由	不完整下载或启动上游
uv / PostgreSQL	环境管理、运行状态和产品数据	pyproject、迁移、Compose、初始化	PG 服务未运行；单测用显式 SQLite
LangGraph	结构化需求规划与校验循环	实际图节点与条件边代码	依赖缺失，对应测试跳过
Temporal	跨阶段持久编排、审批等待、重试	SDK workflow/activity/worker/outbox	服务与 SDK 未运行
OpenSpec	规格与变更文档、CLI 验证	文档生成与真实 CLI 门禁命令	文档生成通过；CLI 未执行
LiteLLM	模型网关	HTTP 适配器、网关配置、Ollama 配置脚本	HTTP mock 契约通过；未调用真实模型
ToolHive	MCP 启动、代理、工具过滤	镜像/准备/启动脚本、部署说明	未启动

工具	在项目中的工作	本包实现	本次真实运行情况
Serena	模板符号上下文	MCP initialize/list/call 读取适配器	未调用真实 LSP
diagrams	技术部署图	可信 Python 渲染函数与导出源文件	缺依赖未运行；ER 使用 dot 已运行
Structurizr / C4	系统、容器、组件图	平台/产品 DSL、新版 validate/export helper	DSL 源生成；解析未执行
Coder	可持久人工 IDE	REST 创建工作区、单管理员门禁	HTTP mock 通过；无实际服务
CubeSandbox	临时 MicroVM 验证器	E2B 创建、文件上传、固定命令、回收	无实际 KVM/Cube 环境
OpenHands	下一阶段自由编码候选	独立隔离探针与扩展说明	未接主链路、未运行

这里“集成”分为主链路代码、外部适配器和独立扩展示例三个等级，不把仅有配置或 API wrapper 写成已经全链路接通。

5. 一次运行的数据流

用户创建项目时，保存名称、模板 ID 和第一条消息。以后补充消息只追加到项目聊天记录。启动 run 时，服务器把当时的消息列表冻结到 `rnd_run.request`，并在同一个数据库事务中写入 `start outbox`。这避免“记录保存了但任务没有可靠提交”的丢单窗口。

dispatcher 从 outbox 取任务，以 run UUID 作为 workflow 稳定身份提交 Temporal。重复提交不应启动第二个相同 workflow；接口还用 owner + idempotency_key 限制重试重复创建。outbox 重试优先照顾尝试次数少的项目，避免单条异常长期堵住后续提交。

plan activity 调用 LangGraph: demo 给出固定规格，litellm 调用真实网关，可选先通过 ToolHive/Serena 获取固定模板符号概览。模型返回纯 JSON，由 Pydantic 检查字段、类型、重复名、保留字、外键存在性和无环约束。结构化规格通过后保存 digest，等待人工批准。

批准请求必须包含同一个 digest，所有权必须匹配，未支持项存在时还需要 accept_limitations。该决定经第二类 outbox 发送为 Temporal signal。浏览器断开不会立即丢弃该 workflow；但能否恢复依赖 Temporal 持久数据与 worker 正确配置，不能仅凭网页显示“运行中”推断成功。

generate activity 从干净上游复制并追加业务运行时、schema、Vue 入口、Alembic 迁移、独立 Compose、文档与图。规范在此生成 OpenSpec change 文档；**当前批准对象是 canonical ProjectSpec JSON，不是用户可以任意改写并执行的 OpenSpec Markdown。**

verify activity 检查源码、业务契约、OpenSpec CLI，以及所选的额外 sandbox 检查。package activity 只有收到报告才能打 ZIP，排除密钥、缓存、依赖和符号链接，计算文件清单和整体 SHA，再标记 READY。下载接口再次核对归属和 SHA。

6. 哪些数据是真实来源

对象	当前事实来源	不应混淆为
原始需求	项目消息 + run 冻结快照	模型事后解释
已批准业务规格	run.spec + digest、交付 approved-spec.json	可执行 Markdown 指令
流程持久事件	Temporal 工作流历史	浏览器轮询状态
平台展示状态	PostgreSQL rnd_run/rnd_event	Temporal 每个底层事件的完整替代
模板版本	固定 Git SHA + manifest/receipt	一直变化的 master
交付源代码	run/product 目录和 ZIP 的文件 SHA	聊天里“已完成”一句话
业务数据	产品自己的 PostgreSQL	平台 rnd_project 或模型上下文
ER 图	已批准业务 metadata 的 biz_ 表结构	线上数据库反射、上游全部管理表

Temporal 开发服务为轻量起步使用持久化 SQLite 文件；**这不替代平台和产品使用 PostgreSQL**。如果要求 Temporal 本身也必须 PostgreSQL 持久化，按官方 server 部署改为专用 Temporal PostgreSQL 数据库和 schema 管理，不能把 CLI start-dev 的文件参数写成 PG URL。[S04]

7. 目录导航

ai-rnd-foundation/	
factory/	原创平台代码
api.py repository.py	HTTP、归属检查、事务
workflows.py activities.py	Temporal 跨阶段流程
planner.py providers/	LangGraph 与外部服务适配
generator.py	模板装配与业务源码生成
product_runtime.py	被复制进产品的可信 CRUD 运行时
artifacts.py	OpenSpec、C4、ER、diagrams
validation.py packaging.py	门禁和安全打包
templates/fastapiadmin/	模板清单、架构规则
overlays/platform/	研发平台 Vue 界面
overlays/product/	生成产品 Vue、启动及容器文件
migrations/	平台 rnd_ 数据表迁移
scripts/	首次组装、启动、探针、验收
integrations/	每个外部工具的接入配方
tests/ reports/	测试与明确范围的证据
docs/ architecture/	完整手册与架构源文件
.vendor/	首次运行后下载的干净上游
runtime/	首次组装后的平台及联合环境
data/runs/<uuid>/	运行产物（本机生成，不进 Git）

`.vendor` 不直接二次开发；修改原创 `factory` 或 `overlays`，重新组装 runtime。否则下一次模板复制时会混入不受追踪的修改。生产升级必须记录 overlay 版本，不能只记录上游 SHA。

8. 首版的可用功能与明确不足

可以表达普通台账、资产、分类、父子记录，支持文本、整数、有限浮点、布尔、日期、时间及引用；后端包含搜索分页、字段校验、归属检查和父记录删除限制；前端动态构建表单和列表。一个模型规格最多 8 个实体、每实体 16 个字段；限制可调整，但扩大上限前先评估 UI、复杂度和测试成本。

没有自动完成唯一约束、富业务规则、精确货币 Decimal、库存交易、流程审批、权限矩阵、文件上传、消息推送、设备实时采集、复杂报表、多人协作、自助租户和费用系统。产品业务表的 `owner_id` 隔离也不是完整多租户/RLS 方案。

生成器只创建新项目。对已上线产品增量修改，必须先检查现有数据库与迁移历史，生成新 migration，保留数据与版本；不能在已部署项目重写初始迁移。

图表与架构包的使用

C4 与 ER 分开

C4表达系统上下文、容器、组件。ER表达表、列、主键和外键。Structurizr DSL不是关系数据库建模语言，本包没有把每张表硬称作一个C4容器。[S05]

平台的 `architecture/workspace.dsl` 描述研发平台；产品ZIP的 `architecture/workspace.dsl` 描述生成的软件。产品另有 `er.dot`、`er.svg`、`er.mmd`、数据字典与业务OpenAPI。ER当前来自生成metadata，只覆盖biz_业务表，不声称已读取全部实际数据库。

查看平台C4

```
docker compose --profile architecture up -d structurizr
```

打开 `http://localhost:8080`。使用的是新的Structurizr local工具，仅限本机开发查看；不要把无认证本地查看器直接对外公开。DSL也可以交给兼容的现行Structurizr工具，继续维护C1/C2/C3。

严格验证和导出

平台根目录执行：

```
bash scripts/export_c4.sh architecture
```

脚本先validate，然后输出JSON和Mermaid。传入生成产品architecture目录即可处理产品的DSL：

```
bash scripts/export_c4.sh /你的产品绝对路径/architecture
```

这条命令要实际运行后才能说“DSL解析通过”。本次只生成了源文件和脚本，未运行Structurizr二进制。新版export支持多种格式，SVG/PNG可能需要包含浏览器的playwright镜像，不能以JSON导出成功冒充PNG已生成。[S05]

ER和部署图

产品中已存在ER SVG时可直接用浏览器打开。Graphviz源可这样重渲染：

```
cd /你的产品目录
dot -Tsvg architecture/er.dot -o architecture/er.svg
```

部署图的Python源码位于 `architecture/deployment.py`，用安装diagrams的环境执行即可生成。代码来自可信renderer，不执行LLM提供的Python。要进一步导出PNG，用同一个业务metadata和明确版本的渲染器，不要依靠截图手工改图。[S16]

和代码同步的验收

字段数、类型、必填、主外键应与business_spec.json和迁移一致。引用可空性需看Mermaid和数据字典，不只看DOT箭头。业务OpenAPI只描述新增business-api，完整上游鉴权和管理API应从实际运行应用导出，文件名已明确区分。

如何增加框架、业务能力和 Agent

1. 扩展前先理解“模板”不是 Git 链接

一个模板至少是：真实仓库固定提交、明确许可证、可运行起点、目录/依赖规则、配置问题清单、数据库迁移方式、生成器、图表映射、验证器和独立交付说明。缺任何一部分，都不应该出现在面向用户的“已支持”下拉框里。

本版只有 `fastapiadmin-pg-v1`，模板选择虽已显示在界面，后台仍明确限制此 ID。没有预装 Spring Boot、React 或 Go 假模板。下面是实际开发路径，不是已完成能力清单。

2. 扩展 FastapiAdmin 业务能力：先小后大

最容易的第一步是增加“字符串唯一编码”。需要同时修改 `FieldSpec` 的能力声明、`Pydantic` 校验、`SQLAlchemy metadata`、冻结迁移、异常映射、UI 错误、ER/数据字典以及测试。必须决定唯一范围：全系统唯一还是 `(owner_id, code)` 联合唯一；两者对业务和多用户意义不同。

实施步骤：先复制现有 tests 增加两个同 owner 重复编码冲突的红测试；再增加另一个 owner 是否允许相同编码的测试；修改 migration renderer 和 runtime；执行 PG 真库验收；最后才让模型在 schema 中输出 unique 字段。只改提示词而没有数据库约束不是完成。

第二步可以将外键 ID 输入换成搜索下拉。新增后端受归属约束的引用候选 API，前端只查询自己可访问的父实体。大表必须有分页，不一次把全部记录拉进浏览器，也不能为了展示下拉而取消所有权限限制。

第三步是精确数值。财务金额不应简单用普通 `number/float` 代表精确小数。新增 `decimal` 类型，明确 precision、scale、JSON 编码和数据库 `Numeric`，并加入边界/四舍五入测试。该能力未完成前，支付、结算、财务软件应列为不支持。

3. 增加第二套框架：建议 Spring Boot + Vue

这是扩展练习，不强制选某个商业框架，也不提供未经核验的第三方仓库冒充可用模板。先选择许可证适合、能在你的电脑独立运行的 Java/Vue Golden Repository，记录 Git SHA。Java 版本、Spring Boot 版本、构建工具、Node 版本全部以该模板实际声明与当前官方要求为准。

阶段 A，人工验证空模板。新目录克隆固定提交，先不接平台，完成数据库配置、后端编译、前端构建、登录和一个手写 CRUD 示例。保存原始日志。若这个步骤不通过，不能把问题推给 Agent。

阶段 B，写架构包。在 `templates/springboot-vue/` 新建 manifest 和 ARCHITECTURE.md，写清 Java 包名、controller/service/repository/entity/DTO 的目录、鉴权扩展点、PG 配置、Maven/Gradle 命令、Flyway/Liquibase 迁移、前端路由以及禁止 AI 修改的文件。

阶段 C，把当前硬编码拆成真实 registry，而不是加一个 JSON 后就结束。建议提供下列协议：


```
class TemplateAdapter(Protocol):
    id: str
    def validate_capabilities(self, spec: ProjectSpec) -> None: ...
    def materialize(self, spec: ProjectSpec, destination: Path) -> dict: ...
    def verification_plan(self, product: Path) -> list[TrustedCheck]: ...
    def export_architecture(self, spec: ProjectSpec, product: Path) -> dict: ...
```

这是目标接口草案，目前没有根代码里假装注册未实现的 adapter。TrustedCheck 应指管理员登记的固定命令/镜像，不接受用户或模型提供任意 command 字符串。

阶段 D，逐处改动：factory/api.py 的模板列表、repository.py 的模板校验、generator.py 的模板选择、activities.py 传递模板 ID、验证器的选择和模型上下文中的能力集。UI 应从 API 取数据，不自行拼接“Java 已支持”。给 run 保存 adapter 版本，旧 run 必须仍可追溯旧 renderer。

阶段 E，实现 Java renderer。把一个已批准实体转换成 Entity、DTO、Repository、Service、Controller、迁移和 Vue 页面。生成代码只从白名单 schema 出发；原始需求中的引号、换行、路径或命令不能进入可执行模板位置。模板字符串本身放 Git 审核。

阶段 F，换成真正的 Java 验证。至少编译、单测、真实 PostgreSQL 集成测试、前端类型与构建、两用户权限、完整 ZIP 在干净目录启动。Java 项目不能调用本版 Python business_runtime 的测试然后声称验收通过。

阶段 G，试点后启用。创建 template_id=springboot-vue-pg-v1 的新项目，检查下载包中的 pom.xml / Gradle 文件、前端源码、迁移、图和独立说明。只有 CI 和真实运行验收都通过，才把模板状态改为 available。

4. 更换前端：Vue → React 或其他方案

前端不是改一个 package.json。为 React 模板准备自己的黄金仓库、鉴权状态管理、API client、表单组件、路由挂载、类型生成、构建及浏览器测试。后端 ProjectSpec 可以共用，但 UI renderer 和前端验证步骤必须不同。保留同一 JSON/OpenAPI 契约有助于隔离变化。

5. 增加 Agent 执行器

当前规划器可替换模型，但不执行任意源码编辑。下一阶段把功能分为 Planner、Coder、Verifier、Reviewer，而不是让四个 Agent 都拥有同样的主机 shell 和密钥。

推荐的增加顺序：先对一个单文件改动实验 OpenHands SDK；再用独立 workspace 改一个可信模板模块；再执行完整测试；最后把它作为 Temporal activity 接入。integrations/openhands/probe.py 仅是第一个隔离实验。

设计输入：已批准 spec digest、选定任务、最小上下文、只写白名单、固定测试计划、短期模型凭据、预算/超时。设计输出：diff、执行记录、测试证据、未完成项、artifact SHA。不得只返回一个“success=true”。

workflow 建议为 plan → 人工批准 → materialize → code → test → 最多一次有预算的 repair → independent verify → package。当前代码没有 code/repair 这两个节点，需要实际新增 activity、状态、错误处理和测试；不要把现有 generate 改成 shell=True 的 Agent 调用。

选择 OpenHands 是针对可嵌入 SDK、远程 Agent Server、工具及工作区分离的工程判断，不是宣称所有任务都优于 Codex、Claude Agent SDK 或 DeepAgents。企业许可证、模型能力、费用、上下文管理和团队熟悉程度都会影响选择。[S11][S12]

6. 与 Cube 和 Coder 的关系

自由编码 Agent 应进入独立执行环境。OpenHands DockerWorkspace 支持其自己的 Agent Server 协议；Cube 暴露 E2B 兼容接口。两者的 API 不是一回事，须在 Cube 模板里部署合适的 Agent Server，或编写能处理文件、命令、事件和生命周期的 Workspace adapter。

Coder 在需要人介入时提供持久目录和 IDE。人工修改后的代码回平台，应走明确的 Git commit/diff 或 artifact import，不能后台悄悄覆盖已经批准的 run 目录。当前只创建工作区并指导手工 ZIP 导入；未来自动传输需用户身份映射和短期下载凭据。

7. 接 GitHub PR / Actions

先让代码成为受控 Git repository，再增加独立 GitProvider。只给目标仓库所需的 installation token 权限；每 run 创建独立分支；提交前检查没有 .env、私钥、数据库备份；PR 描述写原始需求摘要、未支持项、测试范围和 artifact SHA。

CI 验证是独立裁判。Agent 不应能修改发布门禁或伪造测试结果。不要默认自合并；对迁移、权限、支付等敏感变更保留人工批准。本包不自动创建仓库、PR 或合并；.github/workflows/tests.yml 只是本基础代码的可执行契约测试工作流，未在你的 GitHub Actions 账户运行。

8. 图表怎样扩展才不会与代码脱节

当前 C4 来源于模板架构和业务 schema；ER 来源于相同 business metadata。增加部署组件时更新架构 renderer 和测试，不让 LLM 单独画一张与源码无关的漂亮图。

要导出真正数据库 ER，新增 read-only schema reflection adapter：对经授权的测试数据库运行 SQLAlchemy Inspector 或其他工具，导出表、列、PK、FK、索引，再渲染。清楚区分“设计 ER”“迁移后测试库 ER”“生产库 ER”，标注来源、数据库版本与采集时间。不在网页传入任意数据库 URL，防止 SSRF 和数据外泄。

可增加完整 OpenAPI、接口 Markdown、数据字典、迁移 SQL、依赖 SBOM、ADR、测试矩阵和部署手册。每个导出物都注明来源版本，不把没做的验收写成成功。

9. 模板升级和已上线项目更新

模板仓库升级新提交之前，在新的分支修改 manifest SHA，先跑 contract check，确认应用工厂、鉴权返回值、Vue 导出路由和目录没有变化。合并一次成功构建产生的真实锁文件与镜像摘要，增加 overlay 兼容测试。保留旧模板版本，旧 run 的产物不可原地替换。

已上线产品更新不是再次生成一个新空 ZIP 覆盖目录。应计算现有版本与新 spec 的差异，生成新的 migration，做备份与恢复演练，保留人工改动，完成兼容性测试。推荐以后引入 Copier 或专用模板升级策略，但本版没有隐式实现三方合并。

10. 分阶段完成路线与验收门

阶段	真正要做的工作	达到什么才进入下一阶段
M0	首次在线构建固定 FastapiAdmin 主机	登录、API、Vue、PG、Redis 均在本机成功
M1	demo 完整控制流与干净产品启动	下载、SHA、产品 CRUD、持久化、两用户隔离通过
M2	真实 LiteLLM 模型规格规划	多组不同需求生成不同正确结构，失败不降级
M3	OpenSpec/diagrams/C4 导出验收	CLI 验证、SVG、DSL 解析与业务表一致
M4	选用 ToolHive/Serena/Cube/Coder	每个真实探针和资源回收/归属验收通过
M5	OpenHands 单任务代码修改	隔离、diff 白名单、独立测试、预算和超时通过
M6	第二套框架	不同 renderer/verifier 的新项目干净启动通过
M7	生产化	鉴权/租户/审计/限流/备份/恢复/安全审计通过

阶段是成果门槛，不是时间承诺。不能用一个工具名字或截图代替实测证据。

运行、安全与运维

1. 端口与运行角色

服务	主机本地端口	容器内连接方式	说明
平台	8000	api:8000	/web/ 与 /factory-api
平台 PG	55432	postgres:5432	rnd_ 和上游管理数据
平台 Redis	默认不暴露	redis:6379	上游会话和缓存
Temporal	7233 / 8233	temporal:7233	RPC / 开发 UI，均本地绑定
LiteLLM	4000	litellm:4000	ai profile 才启动
Structurizr local	8080	structurizr:8080	architecture profile
生成产品	8010	app:8010	与平台独立运行
产品 PG/Redis	55433 / 56380	postgres:5432 / redis:6379	多产品需改端口与项目名

容器里的 localhost 是该容器，不是 Windows、WSL 或其他容器。Compose 服务通过服务名连接；浏览器通过主机映射端口访问。Cube 的远端 URL、Coder URL、ToolHive MCP endpoint 又属于另一层，不能把四类地址互换。

2. 数据库与迁移

平台表用 `rnd_` 前缀，与上游表分开；产品表用 `biz_` 前缀。平台使用独立 `rnd_alembic_version`，产品使用 `business_alembic_version`。首版依赖上游首次初始化其管理表，新增平台和业务表则有显式迁移；尚未对上游每次升级完成迁移兼容测试。

不要在运行期间 `create_all()` 代替版本化迁移，不要改已部署的 `0001`。新增字段采用新 revision，先在数据库副本执行升级和回滚/恢复测试。不可逆迁移必须明确备份恢复路线。

本版测试的 SQLite 只用于隔离契约；不证明 PostgreSQL 的并发、事务、索引、排序规则和权限全部正确。全栈验收必须实际使用 PG。

3. 凭据与身份

登录身份来自真实 FastapiAdmin。新增研发 API 使用 `owner_id` 检查项目与 `run` 归属；生成业务数据同样使用服务端身份写 `owner_id`，而不接受浏览器随意传入。当前没有组织共享项目、细粒度 workflow RBAC、PG RLS 或收费多租户。

不要认为模板自带管理员角色就自动获得新业务的完整权限矩阵。新功能的租户和协作权限需要单独设计。Coder provisioning 当前故意只开放给一个配置好的 FastapiAdmin 用户，避免共享服务 token 导致跨用户工作区混用。

平台 `.env`、模型 key、Coder token、Cube key 和 MCP token 不进入生成产品。产品 `init_product.py` 自己产生数据库/Redis/JWT 秘密。对外模型会接收需求文本和可选模板符号上下文；敏感项目必须先确认数据能否外发，并采用脱敏或内部模型。

4. Docker socket 是一个高权限入口

默认 worker 没有 Docker socket，static 路线只做源码与可信业务契约测试。需要 Docker sandbox 时先构建验证镜像：

```
docker build -t ai-rnd-verifier:0.1.0 -f integrations/cube/Dockerfile.verifier .
# 仅限受信任的本地实验：
docker compose -f compose.yaml -f compose.sandbox.yaml up -d --force-recreate worker
```

额外 override 为了访问宿主 Docker daemon 将 worker 以高权限运行并挂 socket。即使子容器配置只读、无网络、CPU/内存/进程限制，父 worker 一旦被攻破仍可能控制主机 Docker。这不是多租户隔离方案。生产应把执行器放独立机器/受控服务或真正的 Cube MicroVM，并移除控制面的 Docker 权限。

5. 流程可靠性与限制

提交使用事务 outbox；执行 ID 和 API 幂等键避免重复任务；审批匹配 spec digest；生成使用文件锁和 staging 原子替换；打包使用原子写入与哈希；生成和打包可有限重试。模型规划 activity 不做无界重试，内部只对 schema 校验失败最多再尝试一次。

当前没有用户取消 API、全局费用预算、每用户并发配额、任务清理 API、复杂补偿和工作流版本迁移机制。不要把 Temporal 组件本身的能力等同于平台已实现这些产品功能。新增长时间自由编码时，需要 heartbeat、取消传播、幂等外部资源 ID 和回收记录。

如果 worker 停止，API 仍可能健康，任务会在 QUEUED 等待。Temporal 的 local dev 持久卷被删除后无法靠平台状态表恢复完整事件历史。保留日志、outbox、数据库和 Temporal 数据，分析一致性后再恢复，不随意手工把 run.status 改成 READY。

6. 本地备份示例

在平台根目录操作；备份文件含业务和用户信息，放到受控目录，不提交 Git。

```
mkdir -p private-backups
chmod 700 private-backups
docker compose exec -T postgres sh -c 'PGPASSWORD="$POSTGRES_PASSWORD" pg_dump -U factory -d factory -Fc' > private-backups/factory.dump
```

备份 ZIP/产物还需要 `data/`；凭据需单独加密保管。Temporal dev SQLite 在服务停止后备份所在卷，或使用一致性备份方法，不在高写入时直接复制一个不一致的数据库文件。不要把简单 tar 与 PG dump 当成已经完成灾备。

恢复演练要在新 Compose 项目、空数据库和不同端口做，先确认目标不是现有生产数据库。检查能否登录、旧 run 能否下载且 SHA 一致、产品业务记录和关系是否还在。只有实际恢复成功才叫备份可用。

7. 生产上线阻塞清单

正式公开服务之前，必须完成：TLS/反向代理、默认账号替换、权限与租户模型、每用户配额、密钥管理、网络出站控制、审计留存、依赖/镜像扫描、真实 PG 和浏览器端测试、产物与数据备份恢复、事件告警和资源回收。

模板现有其他模块（上游定时任务、文件、AI、聊天等）也应纳入审计，不能只审新增 factory 目录。单实例限制和上游模块开关要按当前固定提交文档评估，不能盲目加多个 API 副本而破坏内存连接或定时任务行为。
[S01]

用户需求、外部文档和 MCP 结果一律视为不可信输入。模型可提出规格，不能解除安全策略、修改执行镜像、改变下载路径、读取 .env 或写测试报告。静态约束缩小风险，不是完整形式化安全证明。

故障排查：先定位哪一段失败

构建前的问题

docker: command not found / daemon 无法连接。 检查 Docker Desktop 是否运行、WSL Integration 是否启用，执行 `docker version`。不要同时安装两套守护进程后混用 socket。

git fetch 下载失败。 检查 DNS、代理、证书、GitHub 访问与固定 SHA 是否可取。初始包不包含整个上游，网络是必要条件。不要私自改 master 以绕过失败，否则失去版本与 overlay 兼容保证。

uv 解析冲突或 package not found。 看具体依赖名称和索引。联合环境会保留上游 pin；不得无证据把上游版本降级。查看 `runtime/combined/pyproject.toml`，按官方仓库确认该提交是否要求特定索引/平台。网络镜像缺包和真实依赖冲突不是同一问题。

pnpm lock 不匹配或 vue-tsc 失败。 保留错误文件名，先检查 Node22、pnpm9.15.3 和下载的固定上游是否一致，再区分原上游代码还是 overlay 的 TypeScript 错误。不得删除 vue-tsc 命令来假装构建成功。overlay 在本次环境只做了 TypeScript 语法转译检查，没有运行完整 Vue 编译器。

启动的问题

init-data 或 migrate exited。 exited(0) 是一次性成功；非零才是失败。查看对应日志。权限问题检查 Linux data 目录归属及是否位于支持相应权限的文件系统，不对整个主机目录递归 chmod777。

PostgreSQL password authentication failed。 首次初始化后，容器环境变量改变不会自动修改已有卷中的数据库密码。恢复原 `.env` 或用数据库管理方式更改密码；不要为了修配置立即删卷。

端口被占用。 修改 Compose 左侧主机端口，例如 `127.0.0.1:8002:8000`，容器之间仍访问 `api:8000`。产品默认8010、平台8000，不要改错右侧或服务名。

登录页空白/脚本404。 检查前端 build base 为 `/web/`、dist 已复制到 `backend/dist`，并访问 `/web/` 而不是服务根 `/`。源码中的独立新增路由是 `/web/#/factory`，产品是 `/web/#/business`。

登录成功但新增页401。 检查 Auth.getAccessToken、原会话是否已过期、Redis 是否同一实例、SECRET_KEY 是否改变。不要给新增接口去掉鉴权。诊断 API 的 FACTORY_TOKEN 与上游登录 token 不能混用。

workflows 的问题

一直 QUEUED。 依次查看 worker 日志、Temporal健康、任务队列名、namespace、outbox attempt/error。API health 只证明API+DB。必要时运行 `docker compose exec worker python scripts/probe_integrations.py temporal`，连接通过仍不等于对应队列有 worker。

一直 AWAITING_APPROVAL。 这是等待人工确认，不是卡死。审批 hash 必须匹配当前 run；未支持项需明确接受。错身份、过期页面或重复不一致决定会被拒绝。

GENERATING 失败。 检查固定模板 receipt、上游 contract、Graphviz、diagrams、磁盘空间与目录权限。不要将 `.factory-upstream.json` 手工写成目标SHA来绕过校验。

OpenSpec 验证失败。 默认是严格门禁，读取具体 change、requirement 和 scenario 错误。修改 generator 的文档模板并补测试，不在 UI 中将 failures 强制改passed。独立命令：

```
# 在对应 product 目录中：
openspec validate create-product --type change --strict --no-interactive
```

下载409。 可能尚未READY，或ZIP发生了篡改/损坏。检查状态和哈希，重新生成新的run，不修改数据库中的SHA去掩盖不同文件。

模型与外部工具的问题

LiteLLM401。 区分平台到网关 key 和网关到模型 provider key。configure_model 只设置本地Ollama路径，不代表商业API密钥已配置。

LiteLLM400 / JSON格式不支持。 检查模型是否支持请求中的 response_format、max_tokens 等。先用模型官方接口验证能力。必要时在 provider adapter 中做显式能力映射，并增加测试，不能默默 drop 所有未知参数后忽略行为变化。

模型总是输出设备维修。 检查provider是否仍为demo。demo是固定测试fixture，必须改为litellm且探针通过才会分析真实需求。

Serena连接不上。 检查streamable HTTP `/mcp`、ToolHiveproxyport、worker到主机的网络、token、项目索引、Python语言服务器。只用curl收到某个HTTP状态不足以验证MCP initialize/session/tool call。

Cube连接localhost:3000失败。 worker容器的localhost不是你的LinuxCube节点。填真实私网API地址；检查控制面与数据面各自能否到达、证书及templateid。

Coder创建成功但IDE不能打开。 POST接受不等于provisioner构建成功。查看Coderbuild日志、template、agent连接与持久卷。源码导入目前是手工步骤，不会自动出现在新IDE中。

求助时提供什么

提供失败阶段、命令、退出码、去密后的最后几十行日志、操作系统、固定模板SHA、所用image tag/digest 和复现步骤。不要上传整份.env、access_token、完整数据库或未脱敏需求。优先让AI依据具体文件和测试修复，而不是“全部重写成更简单的版本”。

可以直接交给编程 AI 的任务提示词

使用方法：先在 VS Code/IDEA 中打开本项目，让编程 AI 能读取源码。每次只交一项任务；要求它先列出准备修改的文件，再实现、执行测试并报告真实结果。把 `<占位符>` 换成你的实际信息。不要把 `.env` 或个人 access token 发给 AI。任何“已完成”都必须有你可重跑的命令和结果。

P01：首次构建报错，不允许偷偷更换技术栈

你正在修改 ai-rnd-foundation 0.1.0。请先阅读 README.md、docs/START_HERE.md、docs/VALIDATION_REPORT.md、Dockerfile、scripts/bootstrap.py、scripts/resolve_host.py。目标是修复下面真实构建错误，并保持 FastapiAdmin 固定提交、uv、PostgreSQL、Vue3、LangGraph 和 Temporal 的既有职责。

错误日志：<粘贴去掉密钥后的日志>

复现命令：<命令>

操作系统：<Windows WSL/具体Linux版本>

约束：不得替换成另一个 FastAPI 模板；不得删 vue-tsc、OpenSpec 门禁、鉴权或测试；不得把线上 PostgreSQL 改为 SQLite；不得把失败默认为 demo 成功；不得修改 .env 里的密钥。先判断错误来自网络、上游依赖、overlay、Docker、迁移还是业务运行时。只修改必要文件，增加能捕捉此问题的回归测试。不能运行的环境项请明确记为 not_run。交付：修改清单、原因、补丁、重跑命令、真实结果、仍未验证项。

P02：把硬编码模板拆成可扩展 registry

先阅读 templates/fastapiadmin/manifest.json、ARCHITECTURE.md、factory/api.py、repository.py、generator.py、activities.py、validation.py 和 tests/。把唯一模板选择重构为 TemplateRegistry + TemplateAdapter，但保留现有模板行为和API兼容。

需要覆盖：能力校验、固定提交物化、架构输出、可信验证计划、启动说明、adapter版本。不能让模板清单里的任意字符串直接变成 shell 命令；执行器只接收登记的命令类型和参数。run必须保存template_id、模板SHA和adapter版本快照。旧run必须仍可追溯旧版本。未知模板返回422，未通过验收的模板只可标记experimental，不允许伪装available。

先让原有 FastapiAdmin 测试全部通过，再增加一个测试用虚拟adapter验证分发机制。虚拟adapter只能在tests中使用，不出现在用户可选模板里。不要同时实现Java业务生成；本任务只完成真实扩展协议和回归测试。

P03：添加 Spring Boot + Vue 的 Golden Template

在已经完成TemplateRegistry的基础上增加 springboot-vue-pg-v1。

我已人工验收的仓库是 <仓库URL>，固定提交 <SHA>，Java版本 <版本>，构建工具 <Maven/Gradle>。

先读取该模板的启动、鉴权、目录、迁移、前端路由及测试，不凭记忆猜接口。

生成输入仍是经过审批的ProjectSpec，业务范围先限制为CRUD+无环父子关联。

输出必须包括Java实体/DTO/Repository/Service/Controller、Vue页面、PG迁移、

C4 DSL、ER图和数据字典、独立启动说明。保留原模板许可证和身份认证。

未知或不支持的字段/能力应明确拒绝，不静默丢弃。

验收：后端编译和测试、真实PostgreSQL迁移、Vue类型/构建、两用户数据归属、

错误外键、父子删除约束、干净目录启动下载ZIP。不能用Python模板的测试冒充Java验收。

分小步提交，提供每一步的命令与证据，所有未运行项写not_run。

P04：为 FastapiAdmin 添加唯一编码

阅读factory/schemas.py、product_runtime.py、generator.py、artifacts.py及业务测试。

给FieldSpec增加明确的唯一约束能力，默认唯一范围为(owner_id, field)，而非全局唯一。

同时实现Pydantic schema、PG UniqueConstraint、冻结迁移、重复错误409、Vue提示、

OpenSpec和数据字典输出、模型能力提示。不要只做前端检查。

测试必须包含：同owner重复失败，不同owner可用相同编码，更新为重复值失败，

nullable unique语义，真实PG行为，迁移后约束存在。

给已上线项目生成新迁移，不能重写已执行0001或要求用户删库。

P05：给真实模型规划增加质量评测

阅读factory/providers/llm.py、planner.py、schemas.py。建立不含敏感数据的evals需求集：

资产、图书、货架、巡检、客户台账等至少10组，每组记录期望实体、关键字段与不支持项。

加入恶意需求：要求读取.env、执行shell、跳过审批、更换模板、伪造质量报告。

评测检查JSON/schema有效性、领域覆盖、未支持项召回、是否混入固定demo、调用次数和耗时。

不要求自然语言完全逐字一致。HTTP失败不得转换demo。真实模型评测需显式开关与预算；

默认CI只运行不收费的契约测试。输出带模型版本、模板SHA、提示词版本和真实结果的报告。

不得因为一个模型给出“看起来正确”的JSON就宣称任意需求可完成。

P06：将 OpenHands 加入为可选代码修改 activity

先读docs/EXTENDING.md、integrations/openhands/、workflows.py和activities.py。
目标是在独立sandbox中完成一个已批准小任务，不允许在控制面工作目录直接运行agent。
采用核验后的兼容OpenHands SDK/tools/workspace版本和agent-server不可变镜像。

新增CoderProvider协议，输入包括spec_digest、任务、上下文、write_allowlist、预算和超时；
输出包括diff、命令记录、测试证据和未完成项。禁止改鉴权、策略、CI门禁、已有迁移和密钥。
Agent可以运行工作区测试，但最终Verifier使用只读独立环境及固定测试计划。
最多1次repair，迭代次数和费用有上限，超过上限标失败，不无界重试。

为Temporal实现幂等资源ID、heartbeat、取消/超时回收、重启后的连接或清理。
必须真正导出修改后的代码再进入原有package阶段。不能把Conversation结束当作测试通过。
先做一个单模块用例并给证据，不同时开发并行多Agent和自动合并。

P07：真实连接 ToolHive / Serena

阅读integrations/toolhive/和factory/providers/mcp.py。检查当前安装版本帮助及官方文档。
目标是在不暴露未认证公网服务的条件下，从worker完成MCP initialize/list_tools/只读符号读取。
固定一个模板副本；模板根只读，缓存分离；不要挂载Home、平台data、.env或docker.sock。
服务端和客户端都只允许登记的只读工具。

先验证主机连通，再验证worker容器到代理连通，明确每个host/port是谁的地址。
若loopback不可达，给出私网/防火墙/认证代理方案，不能直接改成0.0.0.0无认证。
报告实际返回的工具名、目标文件路径和索引结果；配置存在不代表healthy。
为下一阶段每项目独立实例写设计，但不要现在共享一个可切换活动项目的实例给多用户。

P08：完善 CubeSandbox 执行器

先阅读integrations/cube/、factory/providers/sandbox.py和Cube当前官方E2B兼容示例。
我已有的受控服务：API地址 <私网地址>，template id <ID>，密钥由环境变量提供，不进入提示词。
先用固定无害命令验证创建、文件上传、stdout/stderr/exit_code和退出销毁。

补充实例幂等映射、超时/取消回收、资源限制、文件大小与压缩炸弹防护、
公网和内网出站策略、拒绝访问metadata/平台PG/Redis。不可verify=False。
若要验证整个产品，需要独立PG/Redis和完整依赖环境，不得把AST检查命名为fullstack。
请分别输出控制面、数据面、TLS、模板和隔离测试证据；无KVM环境时明确未运行。

P09: Coder 自动导入源码，保持用户隔离

阅读factory/providers/coder.py、api.py的/coder接口和integrations/coder/README.md。
当前只允许单个显式管理员创建workspace，ZIP导入是手工的。请实现安全的自动导入步骤。

先设计平台用户与Coder身份绑定，不允许所有租户共用一个全权token。
workspace创建、build成功、agent在线、文件传输、SHA验证必须分阶段且幂等。
使用仅对一个artifact有效、短期、可撤销的下载凭据，不传平台登录token或数据库密码。
只解压到/workspace/<project>，拒绝路径穿越和符号链接；原有目录有改动时不覆盖。

提供双用户越权测试、过期凭据、重复请求、断线重试、工作区删除后资源回收测试。
工作区仅创建成功不能标记source_imported或product_ready。

P10: 真正的 PostgreSQL + 浏览器端验收

请阅读现有reports和tests，注意现有业务测试使用显式SQLite测试库，不代表PG验收。
增加一个独立集成测试环境，启动真实PG、Redis、固定上游API和Vue构建产物。
用两个真实登录用户操作生成产品：新增父子、列表/搜索/分页、编辑、删除约束、
跨用户读写/引用拒绝、刷新持久化、重启持久化。

使用当前可用的Playwright或等效浏览器工具，版本锁定；处理上游登录验证码要采用
明确测试配置/测试账号流程，不在生产代码中绕过鉴权。不要依靠硬编码Bearer测试身份。
保存真实日志、截图、迁移版本和失败报告；失败时CI必须失败。
最后从平台下载ZIP，在全新目录/空数据库运行同样验收，不能只测生成前的工作目录。

P11: 架构图、ER 图与代码一致性

检查factory/artifacts.py、product_runtime.py与generator.py。
使用当前Structurizr统一validate/export命令，不引入已停止维护的Lite/旧CLI为默认方案。
对C1/C2/C3 DSL做真实解析，验证容器、组件关系与产品实际进程/代码一致。
ER从业务metadata生成；若新增真库反射，结果必须标注数据库、采集时间和schema范围。

测试包括中文/引号/换行标签、非法DSL include、主外键、可选引用与索引、
不存在的关系、长标识符、业务表与上游管理表边界。
输出可编辑DSL/DOT/JSON及SVG，并注明每个文件的来源，不把LLM绘图当作真库结构。

P12：发布前审计与版本冻结

这是一个尚未通过生产验收的基础工程。请基于完整源码和本机真实部署做审计，不要只根据README评价安全。重点：鉴权、owner隔离、shared token、文件路径、ZIP包敏感信息、SSRF、Docker socket、MCP工具权限、sandbox出站、默认账户、迁移恢复、依赖/镜像漏洞、模型费用和资源配额。

先列风险及复现，再逐项修复并加测试。生成真正运行后的uv.lock、pnpm锁、镜像digest、模板SHA、overlay版本、SBOM、测试矩阵与备份恢复报告。不得编造CVEs、扫描结果、性能或通过率。未验证项保留not_run并阻止对应生产发布条件。

AI 修改结果怎样验收

先看是否保持用户当前技术栈和边界；再看 diff 是否只改目标文件；再运行它给出的命令；最后检查新增测试真的覆盖失败用例，而不是空断言。让 AI 输出“执行了哪些测试、哪些没有执行、为什么”，比要求它写“完全完成、非常优雅”更有价值。

验证报告 · 本次交付范围

1. 结论

本基础包包含实际可执行实现和测试，但 **尚未在本次交付环境证明：真实 FastapiAdmin 完整构建、真实 PostgreSQL + Redis + Temporal + LangGraph workflow、真实模型与全部外部工具联调、下载产品的干净环境启动。**

本次执行环境为 Linux / Python 3.13.5。基础实现按 Python 3.12 容器构建；两者并非同一套已验证依赖环境。容器环境没有可用 Docker daemon、PostgreSQL、LangGraph、Temporal SDK、OpenSpec、diagrams、MCP SDK 及远端工具凭据，且不能直接拉取整个上游仓库或安装完整依赖。没有通过伪造工具、假服务响应或临时禁用生产约束冒充验收。

2. 本地实际结果

测试命令：

```
python -m pytest --junitxml=reports/junit-local.xml
```

本次结果：**64 passed, 1 skipped**。完整输出和JUnit XML保存在reports目录。跳过项是依赖真实LangGraph的demo图执行；它没有被计为通过。

通过范围：标识符/能力schema校验，保留路由冲突，重复字段/无效外键/循环依赖拒绝，CRUD类型、必填、归属和引用约束，API项目归属、幂等、需求冻结、审批hash与限制确认，输出路径与秘密过滤，文件哈希与下载篡改拒绝，LiteLLM/Coder HTTP mock契约。

竖向产物测试使用一个明确标记的 **测试专用最小上游fixture**，执行了真实业务生成、Graphviz ER渲染、源解析、业务契约、ZIP打包和哈希复验。这个测试fixture不能证明真正FastapiAdmin前端/鉴权/数据库初始化可运行，也不作为用户模板发布。生产生成器要求真实固定上游receipt；不会自动退回fixture。

3. 其他检查

Python源码编译检查、Compose YAML结构解析、Shell脚本语法检查和两个Vue组件中TypeScript脚本的语法转译另存 `reports/static-checks.json`。YAML解析不是docker compose config实际解析；TS语法转译不是vue-tsc或Vue模板编译；源码存在不是Docker镜像已构建。

C4源文件已生成，但本次未运行Structurizr parser。diagrams Python部署图源已实现，本次未执行其依赖库；本包可见的ER SVG由实际Graphviz生成。文档PDF单独经过渲染检查，这与软件运行验收不同。

4. 必须由真实环境补齐的验收矩阵

项目	当前状态	本机应执行的验收
固定上游下载/合同	未运行完整下载	bootstrap获取SHA、contract通过
上游+overlay前端	未运行Vue编译	pnpm frozen install、vue-tsc、Vite build
uv联合依赖解析	未运行	实际生成锁文件并保存
PG/Redis/上游登录	未运行	容器健康、初始化、真实账号登录
Temporal完整控制流	未运行	demo提交→审批→READY→下载
LangGraph真实执行	本地跳过	环境依赖安装后重新运行测试与demo
OpenSpec CLI	未运行	strict validate成功并保存日志
diagrams	未运行	deployment SVG实际产生
Structurizr parser	未运行	validate/export实际成功
真实模型	未运行	不同真实需求+非法输出/超时场景
ToolHive/Serena	未运行	initialize/list/read、作用域/认证验证
Cube	未运行	KVM模板、创建/上传/固定检查/销毁
Coder	未运行	身份、template build、IDE和持久化
干净交付产品	未运行	新目录Compose、真实PG、双用户浏览器验收
生产安全/负载/备份恢复	未运行	独立审计、扫描、配额、实际恢复

5. 真实控制流验收脚本

在已完成真实上游登录后，取你自己的有效登录token，只在终端临时环境变量传入；不要提交Git或贴到聊天。FACTORY_LOGIN_TOKEN 不是 .env 中的诊断token。

```
# 交互输入，不把token明文写在shell历史中：
read -r -s -p 'Your current login token: ' FACTORY_LOGIN_TOKEN
export FACTORY_LOGIN_TOKEN
printf '\n'
uv run python scripts/smoke_e2e.py --accept-demo
unset FACTORY_LOGIN_TOKEN
```

脚本实际调用创建项目、启动run、等待审批、批准、等待下载并检查SHA。成功结果保存 `reports/live-smoke/`。即便该脚本通过，**仍不表示脚本自动构建了下载产品**；它的报告明确标注 `fresh_product_compose` 未运行，需按 `START_HERE` 第6节补测。

6. READY 的含义

本版 `READY / scaffold_ready` 表示：在运行环境中执行的指定门禁通过，可以下载源码骨架。它不表示任意用户需求全部满足，也不表示所有外部工具在这一run中都执行。每个run的`quality.json`应作为判据，不能用页面绿色状态替代完整验收。

7. 不做的承诺

本包没有声称是完全离线镜像包、没有声称11个工具均已实机联调、没有声称使用了真实付费模型、没有声称已在你的GitHub创建PR或运行Actions，也没有提供虚构的吞吐、费用、延迟或部署时间。

外部工具接入附录

这些附录是实际适配代码的配置指南，不代表外部服务已在本次环境运行。

ToolHive + Serena：可执行部署配方，尚未进行真实服务联调

本目录配合 `factory/providers/mcp.py` 使用。不是装好一个容器就宣称“安全的多租户 MCP 已完成”。当前只读取一份固定 FastapiAdmin 模板；没有项目切换、自动语言服务器选择和每用户 MCP 实例池。

1. 准备

在 WSL Ubuntu、项目根目录操作。安装官方 ToolHive CLI 后，执行 `thv version`、`thv run --help`，确认支持 `--tools`、`--proxy-port`、`--target-port` 和 `--volume`。安装入口见手册来源 S08。

```
bash integrations/toolhive/prepare.sh
```

脚本第一次解析 Serena HEAD，把实际 SHA 写入 `locks/serena.ref`，以后复用，不追随最新提交。它创建独立模板副本、Python 3.13 Serena 镜像并初始化索引。这里的 SHA 是你本地首次执行时取得的，不是本包已验证的版本。初始化失败时停止；不要为了继续而伪造 `.serena/project.yml`。检查 `project create --help` 和语言服务器安装日志。

```
bash integrations/toolhive/start.sh
```

ToolHive 在主机 `127.0.0.1:9122` 代理 MCP，Serena 在容器内 9121 提供 streamable HTTP。模板代码只读；只有模板自己的 `.serena` 缓存和服务配置可写。不要挂载 `/`、用户 Home、平台 `.env`、整个 `data/` 或 Docker socket。此示例容器尚未做完整镜像加固，不面向不可信公网用户。

2. 核验而不是只看容器运行

主机测试环境设置 `FACTORY_SERENA_URL=http://127.0.0.1:9122/mcp`，然后用包含 MCP 依赖的 Python 环境执行：

```
uv run python scripts/probe_integrations.py serena
```

成功必须同时完成 `MCP initialize`、`list_tools`、`get_symbols_overview`。脚本失败即未接通，不能降级为“已集成”。固定模板的 Python 文件应在返回符号中可辨认。

Docker worker 内的 `127.0.0.1` 是 worker 自己，不能照抄主机地址。Docker Desktop 对 `host.docker.internal` 与主机 loopback 的可达性因部署方式不同，必须在 worker 内实测。不要一遇到连接失败就把未认证 MCP 暴露到 `0.0.0.0`。使用私有网络接口、明确的防火墙来源限制及 ToolHive OIDC/mTLS 认证代理，或先在同一 WSL 主机运行 worker。若采用 Bearer 认证，设置 `FACTORY_SERENA_TOKEN` 为服务验证的短期凭据；填一个任意字符串不会自动建立认证。

把实际可达的 `/mcp` URL 配置到平台 `.env`，重建 `api/worker` 容器并重新执行探针：

```
docker compose up -d --force-recreate api worker
docker compose exec worker python scripts/probe_integrations.py serena
```

3. 安全边界

ToolHive 负责启动、代理、权限与工具暴露；Serena 提供符号级代码理解。应用内 `allowlist` 是第二层限制，不替代服务器侧过滤。未来增加自由编码时，必须为每个工作区创建独立实例，分离读写工具，不能把一个有活动项目状态的 Serena 实例共享给不相关用户。默认基础流程不选 Serena 仍可运行。

4. 已知未验证事项

本包没有运行 ToolHive 或 Serena，也没有验证此版本 LSP 初始化在只读模板条件下的全部行为。先通过上述真实探针，再启用 UI 的 Serena 选项。运行探针只证明读取接口可用，不代表任意编程语言均已可用。

CubeSandbox：远端 KVM MicroVM 执行适配器

本包通过 `e2b-code-interpreter` SDK 实现 `Sandbox.create`、上传交付 ZIP、固定命令检查、结果回传和 context manager 清理。当前用途是 **source verification**，不是完整 FastapiAdmin + PostgreSQL + Redis + 浏览器测试，也没有自由编码 Agent 自动驻留 Cube 的实现。

运行条件

你需要可运行 KVM 的 Linux 环境，验证 `/dev/kvm` 可用，并按官方 CubeSandbox 文档部署 CubeAPI、控制面、Cubelet、数据面及模板。Windows + AMD CPU 不自动等于 WSL/Docker 内已有可用 KVM。可以把平台放 Windows WSL，把 Cube 放私有 Linux 服务器；默认小规模流程不需要 Kubernetes，也不强制安装 Cube。

Cube 官方 quickstart 使用 `E2B_API_URL`（例如私网节点 3000 端口），不是把普通公网域名随意塞入 SDK 的 `domain` 参数。平台设置：

```
FACTORY_CUBE_API_URL=https://your-private-cube-api.example
FACTORY_CUBE_API_KEY=REPLACE_WITH_A_REAL_KEY
FACTORY_CUBE_TEMPLATE=REPLACE_WITH_A_VERIFIED_TEMPLATE_ID
```

HTTP 仅限受信任隔离私网测试；正式连接使用有效 TLS 证书。不要 `verify=False` 跳过校验。若使用内部 CA，应把该 CA 加入信任链。

模板必须具备什么

官方 sandbox-code 镜像提供 Cube 所需 agent/envd；模板中还应有 Python、可写 `/home/user` 和足够磁盘。平台提交的是 `stdlib` 源码检查任务，所以不依赖安装业务项目的所有依赖。`Dockerfile.verifier` 是 **Docker 验证器镜像**，不能直接当成一个具备 Cube agent/envd 的官方 MicroVM 模板。

官方模板创建命令见 S06；镜像先在你的环境验证并锁定 tag/digest，记录 template ID。不要自动使用教程里的 `latest` 作为生产固定版本。

```
docker compose exec worker python scripts/probe_integrations.py cube
```

此探针会实际创建临时 `sandbox`、执行固定的 `Python` 版本检查并销毁，会消耗你的 `Cube` 资源。确认 endpoint、配额和模板后再运行。随后在 UI 选择 `cube` 验证一个示例产物，查看质量报告中的 `cube` 结果。

隔离并非一句 `allow_internet_access=False`

适配器禁用公网出站、设置超时并在退出时回收；但公网出站禁用不等于所有内网资产都不可达。`Cube` 管理员还应限制私网网段、平台 `PG/Redis/metadata` 地址和文件大小，设置资源/数量配额。未通过这些隔离验收前，不把它开放给不可信租户。

Coder：长期 IDE 工作区，不是生成流水线的危险代码沙箱

本包已实现真实 REST 创建工作区适配器：`factory/providers/coder.py`。需要你已部署的 `Coder`、有效 API token、已验证的 template ID。平台不会自动部署 `Coder`，不会替你创建管理员，不会自动配置 `Docker provisioner`，也不会自动把 ZIP 上传到工作区。

配置与验收

先按官方 `Docker` 安装文档部署独立 `Coder`（手册 S07）。`Coder` 使用自己的数据库和生命周期，不能把平台的 `factory` 数据库当成 `Coder` schema。先在 `Coder` 界面手工创建并打开一个可用模板：可以是 `Linux Docker workspace`，也可以是你已有的虚拟机模板。其持久卷必须只属于这个工作区。

在平台 `.env` 配置：

```
FACTORY_CODER_URL=https://your-private-coder.example
FACTORY_CODER_TOKEN=REPLACE_WITH_A_SCOPED_TOKEN
FACTORY_CODER_TEMPLATE_ID=REPLACE_WITH_A_VERIFIED_TEMPLATE_UUID
FACTORY_CODER_OWNER_ID=REPLACE_WITH_YOUR_FASTAPIADMIN_USER_ID
```

`FACTORY_CODER_OWNER_ID` 是 **FastapiAdmin** 中的用户 ID，不是 `Coder` 用户 UUID。只允许这名明确指定的管理员使用当前共享 `Coder` token。否则所有平台用户会在同一个 `Coder` 账号下创建资源，这是本包主动禁止的行为。

```
docker compose up -d --force-recreate api worker
docker compose exec api python scripts/probe_integrations.py coder
```

探针只 GET 当前用户，验证身份，不创建资源。完成一个 READY 项目后，点击“创建 Coder 工作区”，适配器实际 POST `/api/v2/users/me/workspaces`。服务接受请求不等于工作区构建成功，要在 Coder 检查 build 日志和最终状态。重复点击同名工作区可能得到冲突，请在 Coder 找到已创建工作区，不要无限重复创建。

下载平台交付 ZIP，在 Coder 浏览器 IDE 中手工上传、解压，按 `README_DELIVERY.md` 启动。初次验收必须在工作区关闭后重新打开，确认源代码和数据库是否按你选择的模板持久化。

后续自动导入

新增专用 artifact transfer adapter：凭用户绑定的 Coder 身份、一次性短期下载凭证和项目 SHA，在指定工作区只写 `/workspace/<project>`。不能把平台全权 token、宿主 Docker socket、平台数据库密码放进工作区。当前实现不包含这段自动传输；扩展提示词见手册。

OpenHands SDK：下一阶段的自由编码执行器候选

当前主链路的 Agent 是 LangGraph 编排的受约束规格规划器；真实模型通过 LiteLLM 接入。代码由确定性模板生成，因而可检查、可限制且无需允许模型执行任意 shell。

OpenHands SDK 适合作为“根据通过审批的任务改代码、运行测试、修复”的扩展候选：其 Agent、Conversation、DockerWorkspace/remote agent server 分工适合平台嵌入。官方 SDK 也有 OpenAI-compatible endpoint 配置。它不是本包已联调的默认生成引擎，也不是 Cube 的 E2B SDK 的直接替代品。要把 OpenHands 接到 Cube，需要真正的 Workspace/Agent Server 传输适配，而不是把 URL 替换一下。

`probe.py` 是 **可执行的独立隔离示例**，不接收平台工作区，不读取平台 `.env`，不向主业务返回假完成状态。它要求你锁定一组兼容的 SDK/tools/workspace 版本与 agent-server 镜像 digest，并采用一个允许实验的低权限模型 key。建议在专用 Linux 开发机执行，而非直接在生产控制面执行。

```
## 在 integrations/openhands 新建独立 uv 环境；版本取你核验过的同一发行组。
uv init --bare --python 3.13
uv add openhands-sdk openhands-tools openhands-workspace
## 首次安装会实际解析版本：检查 lock 与官方 server 兼容性后保存。
## export OPENHANDS_SERVER_IMAGE=ghcr.io/openhands/agent-server@sha256:<verified-digest>
## export LLM_MODEL=<your-compatible-model>
## export LLM_BASE_URL=<gateway-accessible-from-this-runtime>
## export LLM_API_KEY=<short-lived-scoped-key>
uv run python probe.py
```

该示例会真实启动 Docker Agent Server，并让模型执行一个独立临时目录任务，最多 8 次迭代。DockerWorkspace 的基础容器隔离不是多租户安全证明：仍须处理网络、挂载、凭据、资源配额和命令策略。接入主平台前增加 diff 白名单、不可写测试/策略、独立测试裁判、产物导出、超时回收、Temporal 幂等与审批。详见完整手册的扩展路线和提示词。

官方来源与版本核对

资料核对基线：2026-09-08。以下用于确认工具职责、公开接口和版本；它们不是本平台已完整联调的证明。代码中的组合架构、状态机、边界、脚本和测试由本包实现。网络资料可能更新，首次部署后应保存实际版本和摘要。

- [U01] 用户旧规划：AI_SOFTWARE_RND_PLATFORM_MASTER_PLAN(2).md。参考Golden Template、Architecture Pack、Provider和确定性验收原则；当前请求优先。
- [S01] FastapiAdmin官方仓库与固定提交：<https://github.com/fastapiadmin/FastapiAdmin/tree/f7f5fb61a5c918016640f6b07e053c807381b7cc>。已核对backend/app/init.py、core/dependencies.py、pyproject.toml、frontend/web/package.json、router/index.ts、utils/auth/index.ts及环境样例。当前公开README：<https://github.com/fastapiadmin/FastapiAdmin>。注意不是另一个名为fastapi-admin的TortoiseORM项目。
- [S02] OpenSpec官方仓库、CLI、1.12.0包定义：<https://github.com/Fission-AI/OpenSpec>；<https://github.com/Fission-AI/OpenSpec/blob/main/docs/cli.md>；<https://github.com/Fission-AI/OpenSpec/blob/main/package.json>。
- [S03] LangGraph官方仓库：<https://github.com/langchain-ai/langgraph>。
- [S04] Temporal官方仓库与CLI开发服务：<https://github.com/temporalio/temporal>；<https://docs.temporal.io/cli/setup-cli>；<https://github.com/temporalio/cli/releases>。开发CLI镜像使用1.8.3；镜像是否可在你的registry拉取仍需实测。
- [S05] Structurizr统一工具与DSL：<https://docs.structurizr.com/dsl>；<https://docs.structurizr.com/binaries>；<https://docs.structurizr.com/local>；<https://docs.structurizr.com/validate>；<https://docs.structurizr.com/export>。已核对2026.06.28系列二进制、local/validate/export与旧Lite/CLI停止维护说明；本包用local而非需要另行评估许可的协作server。
- [S06] CubeSandbox官方仓库及E2B示例：<https://github.com/TencentCloud/CubeSandbox>；<https://github.com/TencentCloud/CubeSandbox/tree/master/examples/code-sandbox-quickstart>。已核对E2B_API_URL、e2b-code-interpreter、Sandbox.create、allow_internet_access、files/commands路径与KVM前提。
- [S07] Coder官方安装与工作区API：<https://github.com/coder/coder>；<https://coder.com/docs/install/docker>；<https://coder.com/docs/reference/api/workspaces>。
- [S08] ToolHive官方仓库与thv run参数：<https://github.com/stacklok/toolhive>；https://docs.stacklok.com/toolhive/reference/cli/thv_run/。已核对streamable-http、tools过滤、volume、proxy-port、target-port、host及网络/认证相关参数。
- [S09] LiteLLM官方仓库及代理部署：<https://github.com/BerriAI/litellm>；https://docs.litellm.ai/docs/proxy/docker_quick_start。镜像main-stable是可变标签，必须在本机验证后锁digest。
- [S10] Serena官方仓库与运行说明：<https://github.com/oraios/serena>；https://oraios.github.io/serena/02-usage/020_running.html。核对Python3.13、uvx、MCP服务、project create/index/health-check及项目作用域。

- [S11] OpenHands SDK与Docker Agent Server: <https://docs.openhands.dev/sdk/arch/overview> ; <https://docs.openhands.dev/sdk/getting-started> ; <https://docs.openhands.dev/sdk/guides/agent-server/docker-sandbox> 。本包建议作为第二阶段自由编码候选，不声称已替你完成其生产适配。
- [S12] LangChain DeepAgents官方文档: <https://docs.langchain.com/oss/python/deepagents/overview> 。可作为后续编排/上下文管理比较对象，本包未引入其运行依赖。
- [S13] Microsoft WSL安装与开发环境: <https://learn.microsoft.com/en-us/windows/wsl/install> ; <https://learn.microsoft.com/en-us/windows/wsl/setup/environment> 。
- [S14] Docker Desktop WSL2后端: <https://docs.docker.com/desktop/features/wsl/> 。
- [S15] Astral uv安装与版本: <https://docs.astral.sh/uv/getting-started/installation/> ; <https://github.com/astral-sh/uv/releases> 。构建工具锁定0.12.10；业务依赖的完整锁须实际解析。
- [S16] mingrammer diagrams与Graphviz前提: <https://github.com/mingrammer/diagrams> ; <https://diagrams.mingrammer.com/docs/getting-started/installation> 。

本包不内置第三方字体文件，不重新许可第三方源代码。具体许可证和供应链审计见
THIRD_PARTY_NOTICES.md。